

WORKING WITH STRINGS AS DATA

Cleaning, slicing, searching, and formatting text

```
file_name = "  
FY2026_Equipment_Status_FINAL.CSV  "  
  
normalized = file_name.strip().lower()  
extension = normalized[-3:]  
display = normalized.replace("_", "  
").title()
```

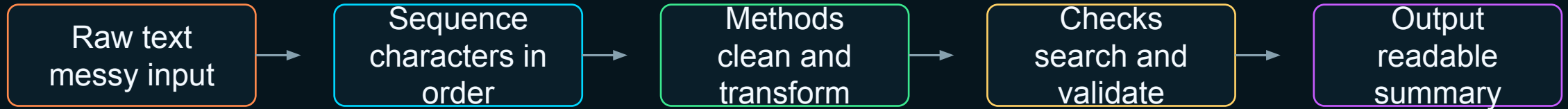
Original: FY2026_Equipment_Status_FINAL.CSV
Normalized: fy2026_equipment_status_final.csv
Display: Fy2026 Equipment Status Final

**Text is data. Clean text is useful
data.**

30 min instruction • 30 min guided lab • filename analysis challenge

The big idea: strings are structured data

A string can be inspected, sliced, cleaned, searched, and displayed.



```
" aLEX joHNSON " → "Alex Johnson"  
"Quarterly_Report.CSV" → "quarterly_report.csv"
```

Strings as sequences of characters

Python can treat text as an ordered sequence, character by character.

```
project_code = "ALPHA-204"
```

Character

A	L	P	H	A	-	2	0	4
0	1	2	3	4	5	6	7	8

Index

Indexing starts at zero. That feels odd at first, then becomes normal.

Indexing: positive and negative positions

Positive indexes count from the left. Negative indexes count from the right.

```
print(project_code[0])  
print(project_code[6])  
print(project_code[-1])
```

A

index 0: first character

2

index 6: seventh character

4

index -1: last character

Negative indexing is handy when the useful part is at the end, such as file extensions.

Slicing: select a range

A slice uses `text[start:end]`. The ending position is not included.

```
project_name = project_code[0:5]  
project_number = project_code[6:9]
```

```
print(project_name)  
print(project_number)
```

ALPHA

204

0 1 2 3 4 5 6 7 8 9

A	L	P	H	A	-	2	0	4
-----						-----		
[0:5]						[6:9]		

The stop point is the boundary after the last character you want.

Common string methods

Methods are built-in actions that create useful string results.

`.strip()`
remove outer spaces

`.lower()`
lowercase text

`.upper()`
uppercase text

`.title()`
display case

`.replace()`
swap text

`.find()`
locate text

`.count()`
count matches

`.isdigit()`
all digits?

```
name = " jordan smith "  
clean_name = name.strip().title()
```

Most string methods return a new string value.

Whitespace and case normalization

Computers treat inconsistent spaces and capitalization as different values.

ACTIVE

Active

active

active

active

```
status = " Active "  
normalized_status = status.strip().lower()  
print(normalized_status)
```

active

Normalization turns messy input into consistent data your program can compare.

String immutability: methods do not edit in place

A string method returns a result. Save that result if you want to keep it.

```
name = " jordan smith "  
name.strip()  
  
print(name)
```

jordan smith

returned value was ignored

```
name = " jordan smith "  
name = name.strip()  
  
print(name)
```

jordan smith

returned value was assigned

Searching and checking within strings

String methods can answer yes/no questions or find positions.

```
file_name = "quarterly_report.csv"

print(file_name.endswith(".csv"))
print(file_name.startswith("quarter"))
print(file_name.find("report"))
print(file_name.count("r"))
```

```
True
True
10
3
```

validate file type

check prefixes

locate text

count occurrences

Text data can be structured information

A filename, ID, or code often contains multiple pieces of information packed into one string.

```
employee_id = "emp-00427"
```

```
prefix = employee_id[0:3]
```

```
employee_number = employee_id[4:]
```

emp

prefix

00427

employee number

A string may look like plain text, but it can carry fields, codes, categories, dates, and extensions.

Formatted output: turn cleaned data into a report

After cleaning and extracting, use f-strings to present results clearly.

```
project_name = "Network Modernization"  
progress = 75.0  
status = "active"  
  
print(f"Project: {project_name}")  
print(f"Progress: {progress:.1f}%")  
print(f>Status: {status.title()}")
```

```
Project: Network Modernization  
Progress: 75.0%  
Status: Active
```

Readable output is part of the program, not an afterthought.

Simple debugging: string mistakes have clues

Read the final line of the error message and look for the first broken assumption.

```
file_name = "report.csv"  
print(file_name[50])
```

IndexError: string index out
of range

```
employee_id = emp-00427
```

NameError or SyntaxError

```
print("ID: " + 427)
```

TypeError

Demo: clean a filename step by step

Start messy. Clean one operation at a time. Print along the way.

```
file_name = "  FY2026_Equipment_Status_FINAL.CSV  "

trimmed = file_name.strip()
normalized = trimmed.lower()
is_csv = normalized.endswith(".csv")
extension = normalized[-3:]
display_name = normalized.replace("_", " ")
                    .replace(".csv", "").title()
```

```
print(f"Original: {file_name}")
print(f"Normalized: {normalized}")
print(f"Extension: {extension}")
print(f"CSV file: {is_csv}")
print(f"Display name: {display_name}")
```

```
Extension: csv
CSV file: True
Display name: Fy2026 Equipment Status
Final
```

Guided exercise: clean inconsistent text values

Students normalize several messy strings, inspect length, extract parts, and format a summary.

```
employee_name = "  aLEX joHNSON "  
department = "engineering "  
status = " ACTIVE"  
employee_id = "emp-00427"  
file_name = "Quarterly_Report.CSV"
```

display original + cleaned values

print lengths before and after

extract employee number

check .csv and extension

replace underscores

create a summary

Lab rule: predict the cleaned value before printing it.

Step 1: normalize the employee fields

Clean whitespace, standardize capitalization, and store the cleaned results.

```
clean_employee_name =  
employee_name.strip().title()  
clean_department = department.strip().lower()  
clean_status = status.strip().lower()  
clean_employee_id = employee_id.strip().upper()  
clean_file_name = file_name.strip().lower()
```

```
Alex Johnson  
engineering  
active  
EMP-00427  
quarterly_report.csv
```

Each cleaned variable is a new value with a name that communicates intent.

Step 2: compare lengths before and after cleaning

`len()` helps reveal invisible whitespace and validate transformations.

```
print(len(employee_name))  
print(len(clean_employee_name))  
  
print(len(department))  
print(len(clean_department))
```

15	→	12
12	→	11

Spaces count as characters. If a value looks right but compares wrong, check for whitespace.

Step 3: extract useful parts from an ID

Use slicing to split a structured string into meaningful pieces.

```
employee_id = "emp-00427"

prefix = employee_id[0:3]
employee_number = employee_id[4:]

print(f"Prefix: {prefix}")
print(f"Employee number: {employee_number}")
```

Prefix: emp

Employee number: 00427

Challenge: what changes if IDs sometimes have different lengths?

Step 4: inspect the filename

Check the extension, extract it, and make the display name easier to read.

```
clean_file_name = file_name.strip().lower()

is_csv = clean_file_name.endswith(".csv")
extension = clean_file_name[-3:]
display_name = clean_file_name.replace("_", " ")

print(is_csv)
print(extension)
print(display_name)
```

```
True
csv
quarterly report.csv
```

Useful pattern: normalize first, then check and extract.

Step 5: create a formatted summary

Make the cleaned data readable for a human.

```
print(f"Employee: {clean_employee_name}")  
print(f"Department: {clean_department.title()}")  
print(f"Status: {clean_status}")  
print(f"ID: {clean_employee_id}")  
print(f"File: {clean_file_name}")  
print(f"CSV file: {is_csv}")
```

```
Employee: Alex Johnson  
Department: Engineering  
Status: active  
ID: EMP-00427  
File: quarterly_report.csv  
CSV file: True
```

Good output proves your transformation worked.

Applied challenge: filename analysis program

Build a program that turns one messy filename into a clean analysis report.

remove whitespace

convert to
lowercase

count characters

check CSV

extract extension

replace underscores

display name

```
file_name = "  FY2026_Equipment_Status_FINAL.CSV  "
```

Build it step by step. Print intermediate values when something looks wrong.

Challenge starter code

Students fill in the transformation lines and print the final report.

```
file_name = " FY2026_Equipment_Status_FINAL.CSV "
```

```
original_display = file_name.strip()
normalized_file_name = file_name.strip().lower()
character_count = len(normalized_file_name)
is_csv = normalized_file_name.endswith(".csv")
extension = normalized_file_name[-3:]
display_name = normalized_file_name.replace("_", " ")
                .replace(".csv", "").title()
```

```
print(f"Original: {original_display}")
print(f"Normalized:
{normalized_file_name}")
print(f"Characters:
{character_count}")
print(f"Extension: {extension}")
print(f"CSV file: {is_csv}")
print(f"Display name: {display_name}")
```

Extension: improve the program so it works for .xlsx or .txt too.

Challenge sample output

Use this as the answer reveal after students build and run their own version.

```
Original: FY2026_Equipment_Status_FINAL.CSV  
Normalized: fy2026_equipment_status_final.csv  
Characters: 33  
Extension: csv  
CSV file: True  
Display name: Fy2026 Equipment Status Final
```

Ask: which lines produce strings, which produce numbers, and which produce Boolean values?

Why this matters: text cleaning is everywhere

Strings show up in filenames, IDs, logs, spreadsheets, web data, APIs, and user input.

files
report_final.csv

IDs
emp-00427

logs
ERROR
2026-07-17

APIs
status: active

forms
Jordan Smith

Today's string tools become tomorrow's data-cleaning tools.

Knowledge check: explain it plainly

Students should leave able to inspect, clean, extract, and format string data.

Why does indexing start at 0?

What does `text[start:end]` mean?

Why does `name.strip()` not change `name` automatically?

Why normalize status values before comparing them?

How can you check whether a filename ends in `.csv`?

What makes a filename structured text data?

Good answer style: “I can show the original, the cleaned value, and the rule I used.”